KTÜ OF TEKNOLOJI FAKÜLTESI · YAZILIM MÜHENDISLIĞI BÖLÜMÜ

Nesne Yönelimli Programlama

2026–2027 Güz Dönemi · Öğrenci Ders Notu

HAFTA 01

Giriş ve Temel Kavramlar

Nesne yönelimli programlamaya genel bakış · Java ile ilk adımlar · Sınıf, nesne, özellik ve metot

ÖZET

Bu hafta, C dilinde edindiğiniz yapısal programlama bilgisinin üzerine nesne yönelimli düşünme biçimini inşa etmeye başlıyoruz. Önce yapısal yaklaşımın büyüyen yazılımlarda neden yetersiz kaldığını somut bir örnek üzerinde göreceğiz; ardından Java çalışma ortamını kurup ilk programımızı yazacağız. Haftanın sonunda **sınıf**, **nesne**, **özellik** ve **metot** kavramlarını tanımlayabilecek ve kendi sınıflarınızı yazabileceksiniz.

Ders	Nesne Yönelimli Programlama (Zorunlu)
Program / Sınıf	Yazılım Mühendisliği · 2. Sınıf · Güz
Haftalık Yük	2 saat teori + 2 saat laboratuvar (toplam 4 saat)
Öğretim Elemanı	Dr. Öğr. Üyesi Rıfat BENVENİSTE
Laboratuvar Sorumlusu	Arş. Gör. Betül MUMCU
Programlama Dili	Java (JDK 21 LTS)
Geliştirme Ortamı	Visual Studio Code + Extension Pack for Java

İçindekiler

1. Bu Haftanın Yol Haritası 3

1.1. Ön Koşul Bilgi Kontrolü 3

1.2. Haftanın Zaman Planı 3

2. Neden Nesne Yönelimli Programlama? 5

2.1. Yazılımın Karmaşıklık Sorunu 5

2.2. Somut Örnek: Basit Bir Banka Hesabı 5

2.2.1. Yapısal Çözüm 5

2.2.2. Nesne Yönelimli Çözüm 6

2.3. İki Yaklaşımın Karşılaştırılması 8

2.4. Kısa Tarihçe 9

3. Java Dili ve Çalışma Ortamı 10

3.1. Java Nedir? 10

3.2. JDK, JRE ve JVM 10

3.3. Derleme ve Çalıştırma Zinciri 11

3.4. JDK Kurulumu 11

3.4.1. Windows 11

3.4.2. macOS 12

3.4.3. Linux (Ubuntu / Debian) 12

3.4.4. Kurulum Doğrulaması 12

3.5. Visual Studio Code Kurulumu ve Yapılandırılması 12

3.6. Proje Klasörü Düzeni 13

3.7. Terminalden Derleme ve Çalıştırma 13

3.8. Sık Karşılaşılan Kurulum Sorunları 14

4. İlk Java Programı 15

4.1. Programın Anatomisi 15

4.1.1. `public class Merhaba` 15

4.1.2. `public static void main(String[] args)` 16

4.1.3. `System.out.println(...)` 16

4.1.4. Deyimler, Bloklar ve Noktalı Virgül 17

4.2. Yorum Satırları 17

5. C'den Java'ya Sözdizimi Köprüsü 18

5.1. Değişkenler ve İlkel Türler 18

5.1.1. Tür Dönüşümleri 19

5.2. Operatörler ve Kontrol Yapıları 19

5.3. Diziler 19

5.4. Metotlar 20

5.5. C ve Java Arasındaki Yapısal Farklar 21

6. Temel Nesne Yönelimli Kavramlar 22

6.1. Sınıf (Class) 22
6.2. Nesne (Object) 22
6.3. Özellik (Attribute / Field) 22
6.4. Metot (Method) 22
6.5. İlk Sınıfımız: `Ogrenci` 23
6.6. Nesne Üretme ve Kullanma 23
6.6.1. `new` Anahtar Sözcüğü ve Bellek 24
6.6.2. Referans Semantiği 25
6.6.3. `null` Değeri 26
6.7. Nesne Dizileri 26
6.8. İkinci Örnek: `Dikdortgen` 26
6.9. Kütüphane Sınıfları da Nesnedir: `Scanner` 27
6.10. Kavram Sözlüğü 28
7. Dört Temel İlkeye Genel Bakış 29
8. Hata Türleri ve Hata Ayıklama 30
8.1. Derleme Zamanı Hataları 30
8.2. Çalışma Zamanı Hataları 30
8.3. Mantık Hataları 31
9. Kod Yazım Standartları 32
10. Haftanın Özeti 33
11. Alıştırmalar 34
11.1. Kavram Soruları 34
11.2. Kod Okuma ve Hata Bulma 34
11.3. Programlama Alıştırmaları 35
11.4. Öz Değerlendirme 36
Öz Değerlendirme Kontrol Listesi 36
12. Kaynaklar 37
12.1. Ders Kitapları 37
12.2. Çevrimiçi Kaynaklar 37
12.3. İletişim 37

1. Bu Haftanın Yol Haritası

Programlama-I ve Programlama-II derslerinde C dilini kullanarak *yapısal programlama* (structured programming) yaklaşımını öğrendiniz. Değişken tanımlamayı, koşul ve döngü yapılarını, dizileri, fonksiyonları ve işaretçileri biliyorsunuz. Bu bilgi bu dersin temelini oluşturuyor; ancak bu dersin amacı yeni bir sözdizimi öğretmek değil, **problemi çözerken kurduğunuz zihinsel modeli değiştirmektir**.

Yapısal programlamada soru şudur: *"Hangi adımları hangi sırayla gerçekleştireceğim?"* Nesne yönelimli programlamada ise soru şudur: *"Bu problemde hangi varlıklar var, her biri neyi bilir ve ne yapabilir?"* Bu iki soru farklı program yapılarına götürür. Dönem boyunca ikinci soruyu sormayı ve cevabını Java diliyle ifade etmeyi öğreneceğiz.

> **Bu Bölümün Sonunda Öğrenci**
>
> - Yapısal programlama ile nesne yönelimli programlama arasındaki farkı somut bir örnek üzerinde açıklayabilir.
> - JDK ve Visual Studio Code kurulumunu kendi bilgisayarında tamamlayabilir; bir Java programını terminalden derleyip çalıştırabilir.
> - Bir Java kaynak dosyasının temel yapısını (`class`, `main`, deyim, blok) tanımlayabilir ve `Merhaba.java` programını hatasız yazabilir.
> - C dilindeki değişken, koşul, döngü, dizi ve fonksiyon yapılarının Java karşılıklarını kullanabilir.
> - **Sınıf**, **nesne**, **özellik (alan)** ve **metot** kavramlarını kendi cümleleriyle tanımlayabilir ve birbirinden ayırt edebilir.
> - `new` anahtar sözcüğüyle nesne üretebilir, nesnenin alanlarına değer atayabilir ve metotlarını çağırabilir.
> - Referans semantiğini (iki değişkenin aynı nesneyi göstermesi) ve `null` değerini açıklayabilir.
> - Derleme zamanı hatası ile çalışma zamanı hatası arasındaki farkı ayırt edebilir.

1.1. Ön Koşul Bilgi Kontrolü

Aşağıdaki soruların tamamına C dilinde cevap verebiliyor olmalısınız. Cevaplayamadığınız madde varsa, laboratuvar öncesinde ilgili konuyu tekrar etmeniz önerilir.

1. `int dizi[10];` tanımı bellekte kaç bayt yer kaplar ve neden?
2. Bir fonksiyona dizi geçirdiğinizde fonksiyon içindeki değişiklikler çağıran tarafa yansır mı? Neden?
3. `struct` ile bir kayıt türü nasıl tanımlanır? Alanlarına nasıl erişilir?
4. `%d`, `%f`, `%s` biçim belirteçleri `printf` içinde ne işe yarar?
5. `for` döngüsünün üç bileşeni (başlatma, koşul, artırma) hangi sırayla çalışır?

1.2. Haftanın Zaman Planı

Süre	Etkinlik	İçerik

Teori 1. saat	Ders anlatımı	Yapısal ve nesne yönelimli yaklaşımın karşılaştırılması; Java'ya giriş (Bölüm 2 ve 3)
Teori 2. saat	Ders anlatımı + tartışma	İlk Java programı, C–Java sözdizimi köprüsü, sınıf ve nesne kavramları (Bölüm 4, 5 ve 6)
Lab 1. saat	Rehberli uygulama	Ortam kurulumunun doğrulanması, `Merhaba.java`, biçimli yazdırma (Görev 1 ve 2)
Lab 2. saat	Bireysel uygulama	İlk sınıfın yazılması, nesne üretimi, referans semantiği deneyi (Görev 3, 4 ve 5)

Tablo 1.1 — Haftalık 4 saatlik ders yükünün dağılımı

2. Neden Nesne Yönelimli Programlama?

2.1. Yazılımın Karmaşıklık Sorunu

Programlama derslerinde yazdığınız programlar genellikle birkaç yüz satırdı ve tek kişi tarafından, tek oturumda yazılıp bitiriliyordu. Gerçek yazılım projeleri ise yüz binlerce satırdan oluşur, onlarca kişi tarafından yıllar boyunca geliştirilir ve sürekli değişir. Bu ölçekte asıl zorluk algoritma bulmak değil, **karmaşıklığı yönetilebilir tutmaktır**.

Bir yazılımın maliyetinin büyük bölümü ilk yazımında değil, sonraki bakım ve değişiklik aşamalarında ortaya çıkar. Bu nedenle iyi bir program yalnızca *doğru çalışan* program değil, aynı zamanda **anlaşılması, değiştirilmesi ve test edilmesi kolay** olan programdır. Nesne yönelimli programlama, bu üç niteliği sağlamak için geliştirilmiş bir düzenleme yaklaşımıdır.

> **BİLGİ**
> Nesne yönelimli programlama bir *hız* tekniği değildir; bir *düzen* tekniğidir. Amacı programı daha hızlı çalıştırmak değil, programın büyüdükçe kontrolden çıkmasını engellemektir.

2.2. Somut Örnek: Basit Bir Banka Hesabı

Kavramları soyut tanımlarla değil, tek bir problemi iki farklı yaklaşımla çözerek inceleyeceğiz. Problem şu: *Bir banka hesabında sahibin adı ve bakiye tutulacak; para yatırma ve para çekme işlemleri yapılabilecek. Bakiye hiçbir zaman negatif olmamalı.*

Bu problemi önce C'de öğrendiğiniz yapısal düşünceyle, sonra nesne yönelimli düşünceyle çözeceğiz. İki çözümü de Java ile yazacağız ki fark dilden değil, **tasarım kararından** kaynaklansın.

2.2.1. Yapısal Çözüm

Yapısal yaklaşımda veri ile onu işleyen fonksiyonlar birbirinden ayrıdır. Veriyi diziler içinde tutar, işlemleri bağımsız fonksiyonlarla yaparız:

YapisalYaklasim.java

```java
public class YapisalYaklasim {

  // Veri: birbirine paralel diziler. Aralarindaki bag yalnizca indistir.
  static String[] sahipler = new String[100];
  static double[] bakiyeler = new double[100];
  static int hesapSayisi = 0;

  public static void main(String[] args){
    int ali = hesapAc("Ali Yilmaz", 500.0);
    int ayse = hesapAc("Ayse Demir", 1200.0);
```

```
11
12    paraYatir(ali, 250.0);
13    paraCek(ayse, 300.0);
14    durumYaz(ali);
15    durumYaz(ayse);
16
17    System.out.println("--- Kural disi kullanim ---");
18    // HICBIR ENGEL YOK: veriye dogrudan erisilebilir.
19          bakiyeler[ali] = -9999.0;
20    durumYaz(ali);
21  }
```

Kod 1.1 — Yapısal yaklaşımda veri ve işlemler ayrıdır

```
▸ TERMINAL — java -cp bin YapisalYaklasim
Ali Yilmaz   :     750.00 TL
Ayse Demir   :     900.00 TL
--- Kural disi kullanim ---
Ali Yilmaz   :   -9999.00 TL
```

Çıktının son satırına dikkat edin. `paraCek` fonksiyonuna yazdığımız *"bakiye negatif olamaz"* kuralı hiçbir işe yaramadı; çünkü `bakiyeler` dizisine programın herhangi bir yerinden doğrudan erişilebiliyor. Kuralı koyduk, ama kuralın uygulanmasını **garanti edemedik**.

> **SIK YAPILAN HATA**
>
> Bu tasarımda üç yapısal zayıflık vardır:
>
> - **Veri korumasızdır.** `bakiyeler[i] = -9999` yazan herkes veriyi bozabilir; derleyici bunu engellemez.
> - **Kural dağınıktır.** "Bakiye negatif olamaz" kuralını uygulamak isteyen her fonksiyon bu denetimi tekrar yazmak zorundadır. Biri unutursa hata oluşur.
> - **Bağlantı zayıftır.** `sahipler[3]` ile `bakiyeler[3]` aynı hesaba aittir, ama bu bağı yalnızca *indis* kurar. Bir dizi sıralanır, diğeri sıralanmazsa veriler karışır.

2.2.2. Nesne Yönelimli Çözüm

Nesne yönelimli yaklaşımda ise sorulan soru değişir: *"Bu problemde hangi varlık var?"* Cevap açıktır: **hesap**. O hâlde bir `Hesap` sınıfı tanımlarız; hesabın bildikleri (sahip, bakiye) ve yapabildikleri (para yatır, para çek) aynı yerde toplanır:

Hesap.java

```
1 public class Hesap {
2
3   // Ozellikler (alanlar): nesnenin durumu
4   private String sahip;
5   private double bakiye;
6
7   // Yapici metot: nesne olusturulurken cagrilir (Hafta 5'te ayrintili islenecek)
8   public Hesap(String sahip, double baslangicBakiyesi){
```

```
    this.sahip = sahip;
    this.bakiye = (baslangicBakiyesi > 0) ? baslangicBakiyesi : 0.0;
}

// Davranislar (metotlar): nesnenin yapabildikleri
public void paraYatir(double miktar){
    if(miktar <= 0){
        System.out.println("Hata: yatirilacak tutar pozitif olmalidir.");
        return;
    }
        bakiye += miktar;
}

public void paraCek(double miktar){
    if(miktar <= 0 || miktar > bakiye){
        System.out.println("Hata: cekilecek tutar gecersiz.");
        return;
    }
        bakiye -= miktar;
}

public double getBakiye(){
    return bakiye;
}

public void durumYaz(){
    System.out.printf("%-12s : %10.2f TL%n", sahip, bakiye);
}
}
```

Kod 1.2 — Veri ve davranış tek bir sorumlulukta toplanmıştır

`private` anahtar sözcüğü, `bakiye` alanına sınıfın **dışından** erişilmesini yasaklar. Bu yasağı derleyici uygular; unutulması ya da atlanması mümkün değildir. Bakiyeyi değiştirmenin tek yolu, kuralı içinde barındıran `paraYatir` ve `paraCek` metotlarından geçmektir.

NesneYaklasimi.java

```
public class NesneYaklasimi{

public static void main(String[] args){
    Hesap ali = new Hesap("Ali Yilmaz", 500.0);
    Hesap ayse = new Hesap("Ayse Demir", 1200.0);

        ali.paraYatir(250.0);
        ayse.paraCek(300.0);
        ali.durumYaz();
        ayse.durumYaz();

    System.out.println("--- Kural disi kullanim denemesi ---");
    // ali.bakiye = -9999.0;    <-- DERLENMEZ: bakiye private'tir.
        ali.paraCek(999999.0);  // Kural sinifin icinde uygulanir.
```

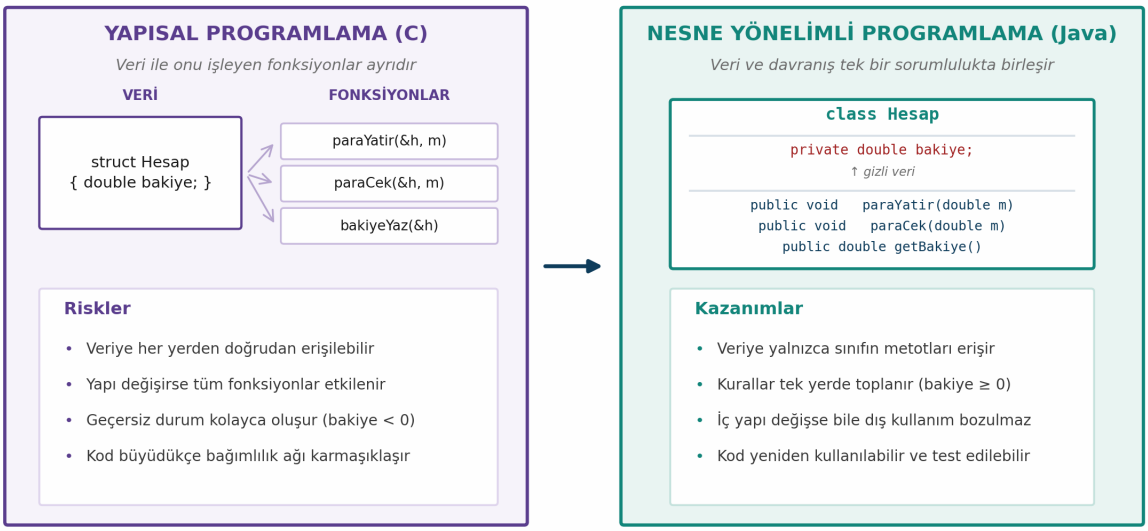
```
15          ali.durumYaz();
16
17      System.out.println("Ali'nin bakiyesi okunabilir: " + ali.getBakiye() + " TL");
18  }
19 }
```

Kod 1.3 — Sınıfın kullanımı

```
▸ TERMINAL — java -cp bin NesneYaklasimi
Ali Yilmaz    :     750.00 TL
Ayse Demir    :     900.00 TL
--- Kural disi kullanim denemesi ---
Hata: cekilecek tutar gecersiz.
Ali Yilmaz    :     750.00 TL
Ali'nin bakiyesi okunabilir: 750.0 TL
```

İPUCU

11. satırdaki `ali.bakiye = -9999.0;` satırının yorum içinde bırakıldığına dikkat edin. Bu satırın yorumunu kaldırırsanız program **derlenmez**. Yapısal çözümde çalışma zamanında sessizce oluşan hata, nesne yönelimli çözümde derleme zamanında yakalanır. Hatanın erken yakalanması, yazılım mühendisliğinin en değerli kazanımlarından biridir.



Şekil 1.1 — Aynı problemin yapısal ve nesne yönelimli çözümlerinin karşılaştırılması

2.3. İki Yaklaşımın Karşılaştırılması

Ölçüt	Yapısal Programlama	Nesne Yönelimli Programlama
Temel yapı taşı	Fonksiyon (işlem)	Sınıf ve nesne (varlık)
Veri ve işlem ilişkisi	Ayrıdır; veri fonksiyonlara parametre olarak taşınır	Birliktedir; veri ve onu değiştiren metotlar aynı sınıftadır

Veri erişimi	Genellikle serbest; her yerden erişilebilir	Denetimli; `private` ile korunur, metotlarla açılır
Kuralların yeri	Kuralı uygulayan her fonksiyonda tekrarlanır	Sınıfın içinde bir kez tanımlanır
Değişikliğin etkisi	Veri yapısı değişirse tüm fonksiyonlar etkilenir	İç yapı değişse de dış kullanım (arayüz) korunabilir
Yeniden kullanım	Fonksiyon kopyalanarak	Miras ve bileşim yoluyla
Ölçeklenme	Kod büyüdükçe bağımlılık ağı karmaşıklaşır	Sorumluluklar ayrıştığı için büyüme daha kontrollüdür
Uygun olduğu durumlar	Küçük, tek amaçlı, kısa ömürlü programlar	Büyük, çok kişili, uzun ömürlü ve değişen sistemler

Tablo 1.2 — Yapısal ve nesne yönelimli programlamanın karşılaştırması

> **DİKKAT**
>
> Yapısal programlama “yanlış”, nesne yönelimli programlama “doğru” değildir. İşletim sistemi çekirdekleri, gömülü sistemler ve yüksek başarımlı sayısal kütüphaneler bugün de büyük ölçüde yapısal yaklaşımla yazılır. Doğru soru *“hangisi daha iyi?”* değil, *“bu problem için hangisi daha uygun?”* sorusudur.

2.4. Kısa Tarihçe

Nesne kavramı 1960'lı yıllarda Norveç'te **Simula 67** diliyle, gerçek dünya benzetimlerini (simülasyon) programlamak için ortaya çıktı. Bir gemiyi, bir kuyruğu ya da bir müşteriyi programda ayrı birer *nesne* olarak temsil etmek doğal bir fikirdi. 1970'lerde Alan Kay ve ekibi **Smalltalk** ile bu fikri bütün bir dil felsefesine dönüştürdü. 1980'lerde Bjarne Stroustrup, C diline sınıf desteği ekleyerek **C++** dilini geliştirdi. 1995'te Sun Microsystems tarafından yayımlanan **Java**, C++'ın sözdizimini sadeleştirip bellek yönetimini otomatikleştirerek nesne yönelimli programlamayı yaygınlaştırdı.

Bugün C#, Python, Kotlin, Swift ve Dart gibi yaygın dillerin tamamı nesne yönelimli özellikler barındırır. Bu haftadan itibaren öğreneceğiniz kavramlar yalnızca Java'ya değil, bu dillerin tümüne aktarılabilir.

3. Java Dili ve Çalışma Ortamı

3.1. Java Nedir?

Java; genel amaçlı, derlenen, güçlü türlü (statically typed) ve nesne yönelimli bir programlama dilidir. Tasarım hedefleri, dilin bugünkü özelliklerini doğrudan açıklar:

Platform bağımsızlığı Kaynak kod bir kez derlenir; oluşan **bytecode** herhangi bir işletim sisteminde, o sistem için yazılmış bir Java Sanal Makinesi (JVM) üzerinde çalışır. Bu ilkeye *"write once, run anywhere"* denir.

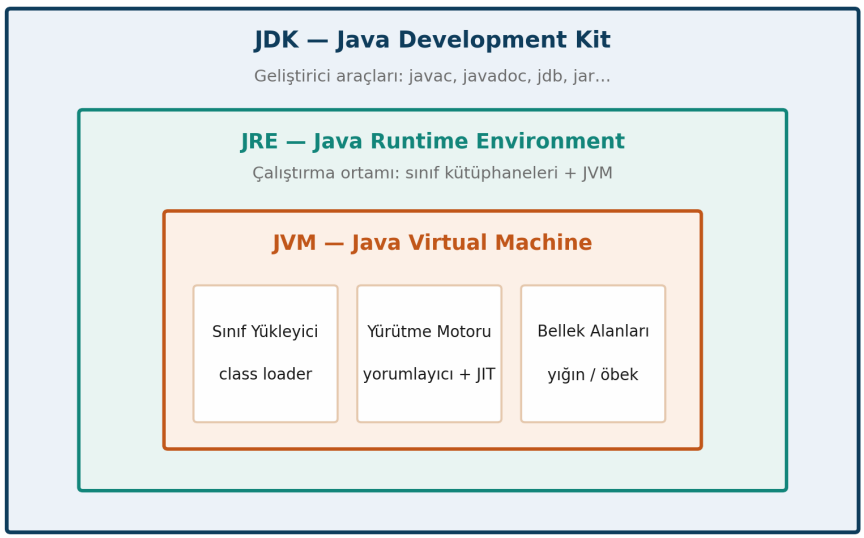
Otomatik bellek yönetimi C'de `malloc` ile ayırdığınız belleği `free` ile geri vermeniz gerekiyordu. Java'da bellek **çöp toplayıcı (garbage collector)** tarafından otomatik olarak geri kazanılır; `free` çağrısı yoktur.

Güvenlik ve sağlamlık İşaretçi aritmetiği yoktur. Dizi sınırları çalışma zamanında denetlenir. Bu sayede C'de sık görülen bellek taşması hataları büyük ölçüde önlenir.

Zengin standart kütüphane Koleksiyonlar, dosya işlemleri, ağ programlama, tarih-saat işlemleri ve çok iş parçacıklı programlama için hazır sınıflar dille birlikte gelir.

3.2. JDK, JRE ve JVM

Java kurulumuyla ilgili üç kısaltma sık karıştırılır. Aralarındaki ilişki iç içe geçmiş kutular gibidir:



Şekil 1.2 — JDK, JRE ve JVM arasındaki kapsama ilişkisi

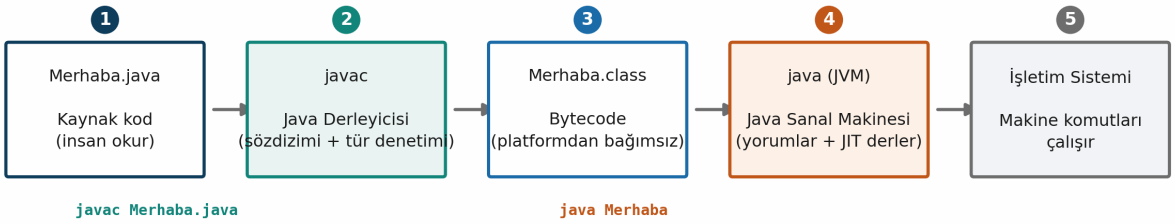
JVM (Java Virtual Machine) Bytecode'u okuyup çalıştıran sanal makinedir. Her işletim sistemi için ayrı bir JVM vardır; programınız için ise tek bir bytecode yeterlidir.

JRE (Java Runtime Environment) JVM ile birlikte standart sınıf kütüphanelerini içerir. Java programını yalnızca *çalıştırmak* için yeterlidir.

JDK (Java Development Kit) JRE'ye ek olarak derleyici (`javac`), belge üreteci (`javadoc`) ve hata ayıklayıcı (`jdb`) gibi geliştirici araçlarını içerir. **Bu derste JDK kurmanız gerekmektedir**; yalnızca JRE yeterli değildir.

3.3. Derleme ve Çalıştırma Zinciri

C'de kaynak kod doğrudan makine koduna derlenirdi ve elde edilen çalıştırılabilir dosya yalnızca derlendiği platformda çalışırdı. Java'da araya bir ara katman girer:



"Bir kez yaz, her yerde çalıştır." Bytecode her platformda aynıdır; JVM her platform için ayrıdır.

Şekil 1.3 — Java kaynak kodundan çalışan programa uzanan zincir

1. `javac Merhaba.java` komutu kaynak kodu **derler** ve `Merhaba.class` adlı bytecode dosyasını üretir. Bu aşamada sözdizimi ve tür denetimleri yapılır.
2. `java Merhaba` komutu JVM'i başlatır; JVM `Merhaba.class` dosyasını yükler, doğrular ve çalıştırır. Sık çalışan kod parçalarını **JIT (Just-In-Time)** derleyicisi çalışma sırasında makine koduna çevirerek hızlandırır.

> **C DİLİNDEN JAVA'YA KÖPRÜ**
>
> C'de `gcc merhaba.c -o merhaba` komutu doğrudan çalıştırılabilir bir dosya üretirdi ve `./merhaba` ile çalıştırırdınız. Java'da `javac` yalnızca bytecode üretir; çalıştırmak için ayrıca `java` komutu gerekir. `java` komutuna dosya adı değil, **sınıf adı** verilir: `java Merhaba` (uzantısız).

3.4. JDK Kurulumu

Derste **JDK 21 LTS** sürümü kullanılacaktır. LTS (Long-Term Support), uzun süreli destek verilen kararlı sürüm anlamına gelir. Daha yeni bir sürüm kurmanız da mümkündür; ancak ders örnekleri JDK 21 ile sınanmıştır.

3.4.1. Windows

1. *https://adoptium.net* adresinden **Temurin JDK 21 (LTS)** Windows x64 `.msi` kurulum dosyasını indirin.
2. Kurulumu başlatın. Bileşen seçimi ekranında **"Set JAVA_HOME variable"** ve **"Add to PATH"** seçeneklerini mutlaka etkinleştirin. Bu adım atlanırsa terminalden `javac` komutu çalışmaz.
3. Kurulum bittikten sonra **yeni bir** Komut İstemi (`cmd`) penceresi açın. Açık olan pencereler eski PATH değerini taşır.

4. `java -version` ve `javac -version` komutlarını çalıştırın. İki komut da sürüm bilgisi yazdırmalıdır.

3.4.2. macOS

1. *https://adoptium.net* adresinden **Temurin JDK 21 (LTS)** macOS `.pkg` dosyasını indirin. İşlemcinize uygun sürümü seçin: Apple Silicon (M1/M2/M3) için **aarch64**, Intel işlemciler için **x64**.
2. Kurulumu tamamlayın; PATH ayarı otomatik yapılır.
3. Terminal uygulamasını açıp `java -version` ve `javac -version` komutlarını çalıştırarak kurulumu doğrulayın.
4. Alternatif olarak Homebrew kullanıyorsanız: `brew install --cask temurin@21`

3.4.3. Linux (Ubuntu / Debian)

```
Terminal
sudo apt update
sudo apt install openjdk-21-jdk
java -version
javac -version
```

Ubuntu üzerinde JDK kurulumu

3.4.4. Kurulum Doğrulaması

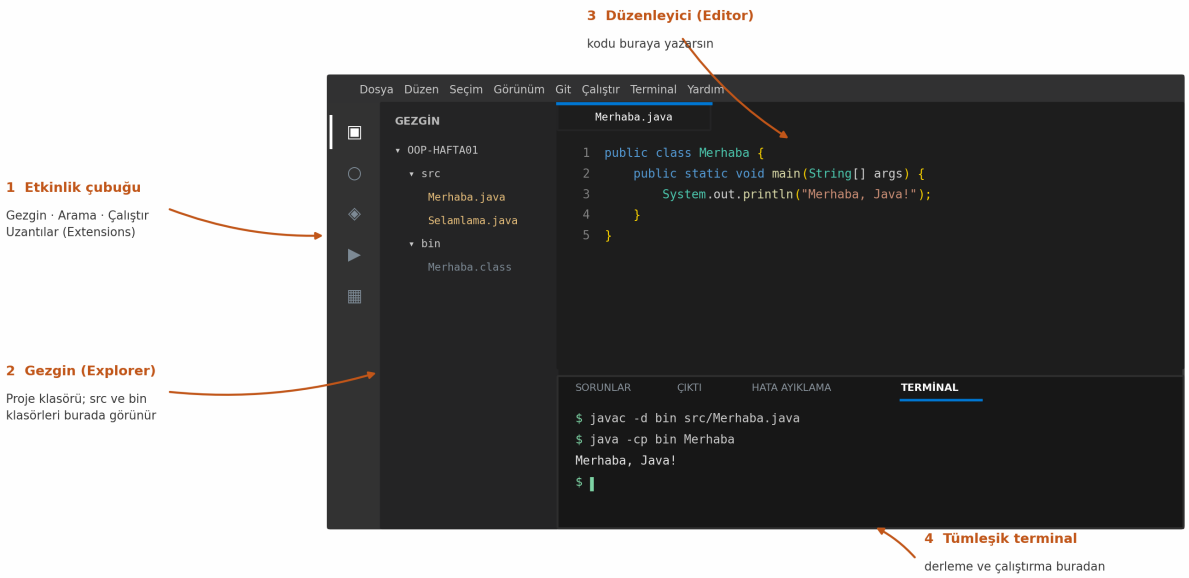
Aşağıdaki iki komutun da sürüm bilgisi yazdırması gerekir. `java` çalışıp `javac` çalışmıyorsa, sisteminizde yalnızca JRE kurulu demektir; JDK kurmanız gerekir.

```
▸ TERMINAL — Kurulum doğrulaması
$ java -version
openjdk version "21.0.5" 2024-10-15 LTS
OpenJDK Runtime Environment Temurin-21.0.5+11 (build 21.0.5+11-LTS)
OpenJDK 64-Bit Server VM Temurin-21.0.5+11 (build 21.0.5+11-LTS, mixed mode)

$ javac -version
javac 21.0.5
```

3.5. Visual Studio Code Kurulumu ve Yapılandırılması

1. *https://code.visualstudio.com* adresinden işletim sisteminize uygun sürümü indirip kurun.
2. VS Code'u açın. Sol taraftaki **Uzantılar (Extensions)** simgesine tıklayın (kısayol: `Ctrl+Shift+X`, macOS'ta `Cmd+Shift+X`).
3. Arama kutusuna **"Extension Pack for Java"** yazın ve Microsoft tarafından yayımlanan paketi kurun. Bu paket; dil desteği, hata ayıklayıcı, test çalıştırıcı ve proje yöneticisi uzantılarını birlikte getirir.
4. Kurulum sonrası VS Code'u yeniden başlatın.
5. `Ctrl+Shift+P` ile komut paletini açıp **"Java: Configure Java Runtime"** komutunu çalıştırın; JDK 21'in tanındığını doğrulayın.



Şekil 1.4 — Visual Studio Code arayüzünün ders boyunca kullanılacak bölümleri

3.6. Proje Klasörü Düzeni

Dönem boyunca her hafta için ayrı bir klasör kullanacağız. Kaynak dosyalar `src`, derlenmiş sınıf dosyaları `bin` klasöründe tutulacaktır. Bu ayrım, profesyonel Java projelerindeki düzenin sadeleştirilmiş hâlidir:

```
●Klasör yapısı

OOP-Hafta01/
├── src/                ← kaynak dosyalar(.java)
      Merhaba.java
      Ogrenci.java
      OgrenciDemo.java
└── bin/                ← derlenmiş dosyalar(.class)
     Merhaba.class
     Ogrenci.class
     OgrenciDemo.class
```

Önerilen klasör düzeni

3.7. Terminalden Derleme ve Çalıştırma

VS Code'da `Ctrl+ö` (ya da **Terminal → Yeni Terminal**) ile tümleşik terminali açın. Proje kök klasöründeyken:

```
▸ TERMINAL — Derleme ve çalıştırma
$ javac -d bin src/Merhaba.java
$ java -cp bin Merhaba
Merhaba, Java!
KTU Of Teknoloji Fakultesi
```

Komut parçası	Anlamı
`javac`	Java derleyicisi

`-d bin`	Üretilen `.class` dosyalarını `bin` klasörüne yaz
`src/Merhaba.java`	Derlenecek kaynak dosya (yol ile birlikte)
`java`	Java Sanal Makinesi başlatıcısı
`-cp bin`	Sınıf dosyalarını `bin` klasöründe ara (classpath)
`Merhaba`	Çalıştırılacak sınıfın adı — **uzantısız** yazılır

Tablo 1.3 — Derleme ve çalıştırma komutlarının bileşenleri

> **İPUCU**
>
> VS Code'da bir `main` metodunun hemen üstünde beliren **Run | Debug** bağlantılarına tıklayarak da programı çalıştırabilirsiniz. Ancak derleme komutlarını en az bir kez elle yazmanız, arka planda ne olduğunu anlamanız açısından önemlidir. Sınavlarda komut satırı bilgisi sorulabilir.

3.8. Sık Karşılaşılan Kurulum Sorunları

Belirti	Olası neden	Çözüm
`'javac' is not recognized...` / `command not found: javac`	JDK kurulu değil ya da PATH'e eklenmemiş	JDK'yı kurun; kurulumda “Add to PATH” seçeneğini işaretleyin; terminali yeniden açın
`java` çalışıyor, `javac` çalışmıyor	Sistemde yalnızca JRE var	JDK sürümünü kurun
`Error: Could not find or load main class Merhaba`	Yanlış klasörde çalıştırılıyor ya da `-cp` verilmemiş	`bin` klasörünün yolunu `-cp` ile belirtin; sınıf adını uzantısız yazın
`class Merhaba is public, should be declared in a file named Merhaba.java`	Dosya adı ile `public` sınıf adı farklı	Dosyayı sınıf adıyla birebir (büyük/küçük harf dâhil) yeniden adlandırın
Türkçe karakterler bozuk görünüyor	Kaynak dosya kodlaması UTF-8 değil	VS Code sağ alt köşeden kodlamayı UTF-8 yapın; derlerken `javac -encoding UTF-8` kullanın
VS Code “Java runtime could not be located” uyarısı veriyor	Uzantı JDK'yı bulamıyor	Komut paletinden **Java: Configure Java Runtime** ile JDK yolunu elle tanımlayın

Tablo 1.4 — Kurulum ve ilk çalıştırma sorunları için hızlı başvuru

4. İlk Java Programı

Şimdi ilk programımızı satır satır inceleyelim. Bu program çok kısa olmasına karşın Java'nın temel yapı taşlarının hemen hepsini içerir.

src/Merhaba.java

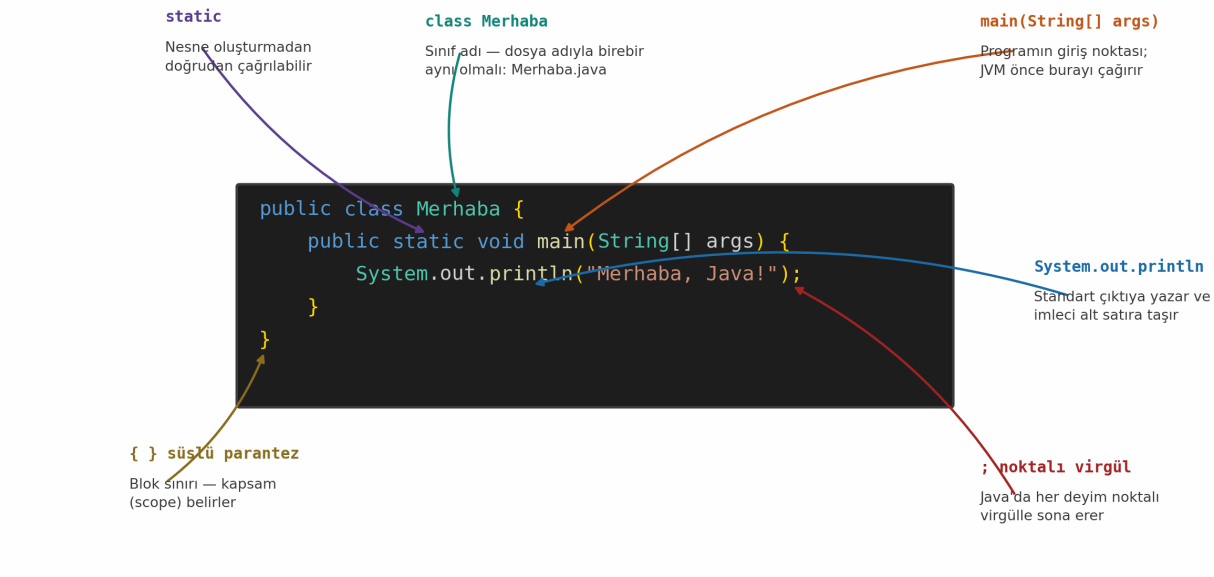
```java
public class Merhaba {

  public static void main(String[] args){
    System.out.println("Merhaba, Java!");
    System.out.println("KTU Of Teknoloji Fakultesi");
    System.out.print("Bu satirin sonunda alt satira gecilmez. ");
    System.out.println("Bu yazi ayni satirda devam eder.");
  }
}
```

Kod 1.6 — İlk Java programı

```
▸ TERMINAL — java -cp bin Merhaba
Merhaba, Java!
KTU Of Teknoloji Fakultesi
Bu satirin sonunda alt satira gecilmez. Bu yazi ayni satirda devam eder.
```

4.1. Programın Anatomisi



Şekil 1.5 — İlk programın yapı taşları

4.1.1. `public class Merhaba`

Java'da **her kod bir sınıfın içinde yer alır**. C'de olduğu gibi sınıf dışında serbest fonksiyon yazamazsınız. `class` anahtar sözcüğü yeni bir sınıf tanımlar; `Merhaba` ise bu sınıfın adıdır. `public` erişim belirleyicisi, sınıfın programın her yerinden görülebileceğini bildirir.

> **DİKKAT**
>
> Bir dosyada en fazla **bir tane** `public` sınıf bulunabilir ve bu sınıfın adı dosya adıyla **birebir aynı** olmalıdır (büyük/küçük harf dâhil). `public class Merhaba` yazan bir dosyanın adı `Merhaba.java` olmak zorundadır; `merhaba.java` ya da `Merhaba1.java` kabul edilmez.

4.1.2. `public static void main(String[] args)`

Bu satır programın **giriş noktasıdır**. `java Merhaba` komutunu verdiğinizde JVM `Merhaba` sınıfını yükler ve tam olarak bu imzaya sahip metodu arayıp çağırır. İmzanın her parçasının bir anlamı vardır:

Parça	Anlamı
`public`	JVM sınıfın dışından çağırabilsin diye herkese açık
`static`	Metot sınıfa aittir; çağrılabilmesi için nesne oluşturulması gerekmez (Hafta 6'da ayrıntılı işlenecek)
`void`	Metot geri değer döndürmez
`main`	JVM'in aradığı metodun adı — değiştirilemez
`String[] args`	Komut satırından programa geçirilen argümanlar; C'deki `argv` dizisinin karşılığı

Tablo 1.5 — `main` metodunun imzasının çözümlenmesi

> **SIK YAPILAN HATA**
>
> `main` metodunun imzasında yapılan en küçük değişiklik bile programın başlamamasına yol açar:
>
> - `static` unutulursa: `Main method is not static in class Merhaba`
> - `String[] args` yerine `String args` yazılırsa metot bulunamaz.
> - `Main` (büyük M) yazılırsa Java bunu farklı bir metot sayar; `main` metodu bulunamadı hatası alınır.

4.1.3. `System.out.println(...)`

`System` standart kütüphanede tanımlı bir sınıftır; `out` bu sınıfın standart çıktı akışını temsil eden alanıdır; `println` ise bu akışa yazan metottur. Yani bu ifade *"System sınıfının out nesnesinin println metodunu çağır"* demektir. Nokta işaretinin okunuşu **"…in …"** şeklindedir.

Metot	Davranışı	Örnek çağrı
`println(x)`	`x` değerini yazar ve alt satıra geçer	`System.out.println("Merhaba");`
`print(x)`	`x` değerini yazar, alt satıra **geçmez**	`System.out.print("Ad: ");`
`printf(bicim, ...)`	C'deki `printf` gibi biçimli yazar	`System.out.printf("%.2f%n", 3.14159);`

Tablo 1.6 — Standart çıktı metotları

> **BİLGİ**
>
> `printf` içinde satır sonu için C'deki `\n` yerine **`%n`** kullanmanız önerilir. `%n` işletim sisteminin satır sonu karakterini üretir; Windows ve Linux arasında taşınabilirliği artırır.

4.1.4. Deyimler, Bloklar ve Noktalı Virgül

Java'da her **deyim (statement)** noktalı virgülle biter. Süslü parantezler `{ }` bir **blok** oluşturur ve kapsam (scope) belirler. Bir blok içinde tanımlanan değişken yalnızca o blok içinde geçerlidir. Bu kurallar C ile birebir aynıdır.

4.2. Yorum Satırları

Yorumlar.java

```java
// Tek satırlık yorum — C99'daki ile aynıdır.

/* Birden fazla satıra
   yayılan yorum. */

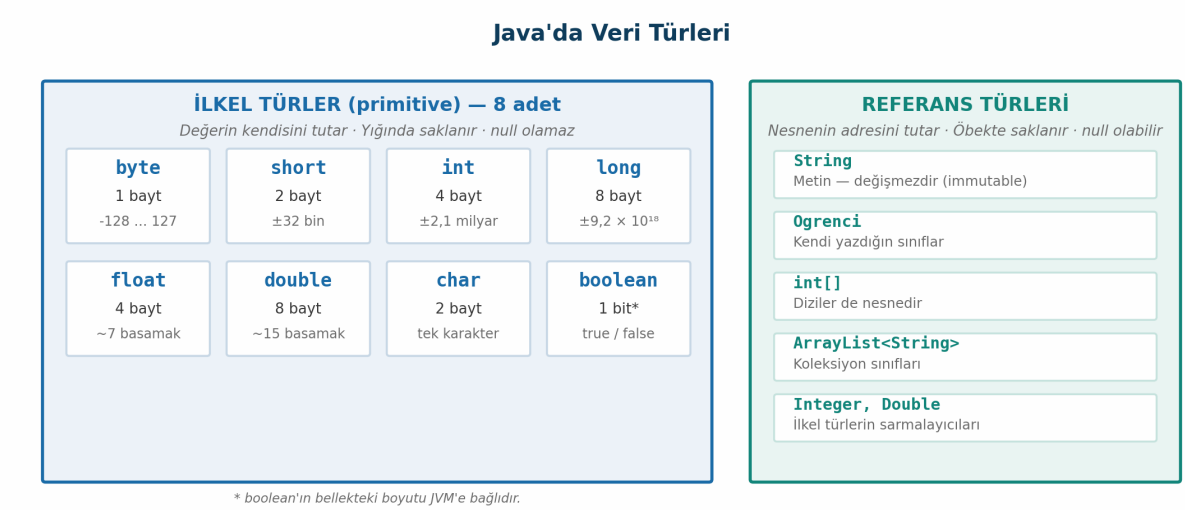
/**
 * Javadoc yorumu. Bu biçimde yazılan yorumlardan
 * javadoc aracıyla otomatik belge üretilebilir.
 * @param x karesi alınacak sayı
 * @return x'in karesi
 */
public static int kare(int x){return x * x; }
```

Kod 1.7 — Java'da üç yorum biçimi

5. C'den Java'ya Sözdizimi Köprüsü

Bu bölüm, C'de bildiklerinizi Java'ya taşımanız için hazırlanmış bir köprüdür. İyi haber şu: kontrol yapıları, operatörler ve temel türler neredeyse birebir aynıdır. Aşağıdaki program, öğrendiğiniz yapıların Java karşılıklarını tek bir yerde toplar.

5.1. Değişkenler ve İlkel Türler



Şekil 1.6 — Java'da ilkel türler ve referans türleri

C'den en önemli farklar şunlardır:

- **Tür boyutları sabittir.** C'de `int` boyutu derleyiciye ve platforma göre değişebilirdi; Java'da `int` her platformda tam olarak 4 bayttır. Bu, platform bağımsızlığının bir gereğidir.
- **`boolean` ayrı bir türdür.** C'de koşullarda 0/1 tam sayıları kullanılırdı. Java'da `if (sayi)` yazamazsınız; koşul mutlaka `boolean` üretmelidir: `if (sayi != 0)`.
- **`String` ilkel tür değildir.** C'de metin `char` dizisiydi. Java'da `String` bir sınıftır; `.length()`, `.toUpperCase()`, `.substring()` gibi metotları vardır ve **değişmezdir (immutable)**.
- **İşaretsiz (unsigned) tür yoktur.** Java'da `unsigned int` bulunmaz.

CdenJavaya.java

```java
// ---- 1) Degisken tanimlama: C ile neredeyse ayni ----
int sayi = 42;
double oran = 3.75;
char harf = 'A';
boolean gecerliMi = true;    // C'de int 0/1 idi, Java'da ayri bir tur
String ad = "Yazilim Muhendisligi";// C'de char[] idi, Java'da bir sinif

System.out.println("sayi      = " + sayi);
System.out.println("oran      = " + oran);
System.out.println("harf      = " + harf);
System.out.println("gecerliMi = " + gecerliMi);
```

```
12    System.out.println("ad         = " + ad);
```

Kod 1.8 — Değişken tanımlama ve yazdırma

5.1.1. Tür Dönüşümleri

Küçük türden büyük türe dönüşüm otomatiktir (**genişleyen dönüşüm**). Büyük türden küçüğe geçerken veri kaybı olabileceği için açık dönüşüm (**cast**) yazmanız gerekir:

Donusum.java

```
1 int a = 7;
2 double b = a;               // otomatik: int -> double
3
4 double c = 3.99;
5 int d = (int) c;            // acik donusum: 3 (ondalik kisim atilir, yuvarlanmaz)
6
7 int toplam = 7, adet = 2;
8 double yanlis = toplam / adet;          // 3.0  <- once tam sayi bolmesi yapilir!
9 double dogru  = (double) toplam / adet; // 3.5  <- once donusum, sonra bolme
```

Kod 1.9 — Tür dönüşümleri ve tam sayı bölmesi tuzağı

> **SIK YAPILAN HATA**
>
> **Tam sayı bölmesi**, C'de olduğu gibi Java'da da en sık yapılan hatalardan biridir. `7 / 2` işleminin sonucu `3`'tür, `3.5` değil. Ortalama hesaplarken bölenlerden en az birini `double`'a dönüştürmeyi unutmayın.

5.2. Operatörler ve Kontrol Yapıları

Aritmetik, karşılaştırma, mantıksal ve atama operatörleri C ile aynıdır: `+ - * / %`, `== != < > <= >=`, `&& || !`, `+= -= *= /= %=`, `++ --`. `if / else`, `switch`, `for`, `while`, `do-while`, `break` ve `continue` yapıları da birebir aynı şekilde çalışır.

CdenJavaya.java

```
1   // ---- 3) Kosul yapilari: C ile birebir ayni ----
2   if(sayi % 2 == 0){
3     System.out.println(sayi + " cift sayidir.");
4   }else{
5     System.out.println(sayi + " tek sayidir.");
6   }
7
8   // ---- 4) Donguler: for, while, do-while ayni ----
9   System.out.print("1'den 5'e: ");
10  for(int i = 1; i <= 5; i++){
11    System.out.print(i + " ");
12  }
13  System.out.println();
```

Kod 1.10 — Koşul ve döngü yapıları C ile aynıdır

5.3. Diziler

Dizilerde sözdizimi biraz farklıdır ve önemli bir kolaylık vardır: **dizi kendi uzunluğunu bilir**. C'de dizi boyutunu ayrı bir değişkende taşımanız gerekiyordu; Java'da `dizi.length` ifadesi bu bilgiyi verir.

İşlem	C	Java
Tanımlama	`int notlar[5];`	`int[] notlar = new int[5];`
İlk değer verme	`int n[] = {1,2,3};`	`int[] n = {1,2,3};`
Uzunluk	ayrı değişkende tutulur	`n.length` (parantezsiz)
Sınır denetimi	yok — taşma sessizce olur	var — `ArrayIndexOutOfBoundsException` fırlatılır
Varsayılan değer	belirsiz (çöp değer)	sayısal türlerde 0, `boolean` için `false`, referanslarda `null`

Tablo 1.7 — C ve Java'da dizilerin karşılaştırılması

CdenJavaya.java

```java
// ---- 5) Diziler: sozdizimi farkli, uzunluk dizide saklanir ----
int[] notlar = {68, 74, 91, 55, 80}; // C: int notlar[5] = {...};
int toplam = 0;
for(int i = 0; i < notlar.length; i++){ // C'de ayri bir boyut degiskeni gerekirdi
        toplam += notlar[i];
}
System.out.println("Not ortalamasi = "+(double) toplam / notlar.length);

// ---- 6) Gelismis for (for-each): C'de karsiligi yok ----
 int enBuyuk = notlar[0];
for(int not : notlar){
  if(not > enBuyuk){
            enBuyuk = not;
  }
}
System.out.println("En yuksek not  = " + enBuyuk);
```

Kod 1.11 — Diziler ve gelişmiş for (for-each) döngüsü

> **BİLGİ**
>
> **Gelişmiş for döngüsü** (`for (int not : notlar)`) C'de karşılığı olmayan bir kolaylıktır. "Dizideki her eleman için" anlamına gelir. İndise ihtiyacınız olmadığında bu biçimi tercih edin; kod hem kısalır hem de sınır hatası riski ortadan kalkar.

5.4. Metotlar

C'deki fonksiyonların Java'daki karşılığı **metotlardır**. Tek fark, metotların her zaman bir sınıfın içinde tanımlanmasıdır. Sözdizimi aynıdır:

```java
●Metotlar.java
1 public static int kare(int x){
2   return x * x;
3 }
4
5 public static double ortalama(int[] dizi){
6   int toplam = 0;
7   for(int deger : dizi){
8         toplam += deger;
9   }
10   return(double) toplam / dizi.length;
11 }
```

Kod 1.12 — Metot tanımlama

Java'da **aşırı yükleme (overloading)** desteklenir: aynı adı taşıyan birden çok metot, parametre listeleri farklı olduğu sürece birlikte var olabilir. C'de bu mümkün değildir.

```java
●Metotlar.java
1 public static int enBuyuk(int a, int b)    {return a > b ? a : b; }
2 public static double enBuyuk(double a, double b){return a > b ? a : b; }
3 public static int enBuyuk(int a, int b, int c) {return enBuyuk(enBuyuk(a,b), c);}
```

Kod 1.13 — Metot aşırı yükleme

5.5. C ve Java Arasındaki Yapısal Farklar

Konu	C	Java
Program birimi	Fonksiyonlar dosya düzeyinde serbesttir	Her kod bir sınıfın içinde yer alır
Bellek yönetimi	`malloc` / `free` ile elle	`new` ile ayrılır; çöp toplayıcı otomatik geri alır
İşaretçi	`*` ve `&` ile adres aritmetiği yapılır	İşaretçi yoktur; **referans** vardır, aritmetiği yapılamaz
Başlık dosyası	`#include <stdio.h>`	`import java.util.Scanner;` — derleyici doğrudan sınıfı bulur
Ön işlemci	`#define`, `#ifdef` makroları	Ön işlemci yoktur; sabitler için `static final` kullanılır
Kayıt türü	`struct` — yalnızca veri	`class` — veri ve davranış birlikte
Metin	`char*` / `char[]`	`String` sınıfı — değişmez (immutable), zengin metot kümesi
Çalıştırma	Derlenen dosya doğrudan işletim sisteminde çalışır	Bytecode JVM üzerinde çalışır

Tablo 1.8 — C ile Java arasındaki temel yapısal farklar

6. Temel Nesne Yönelimli Kavramlar

Bu bölüm, dönemin geri kalanının üzerine kurulacağı dört kavramı tanıtır: **sınıf**, **nesne**, **özellik** ve **metot**.

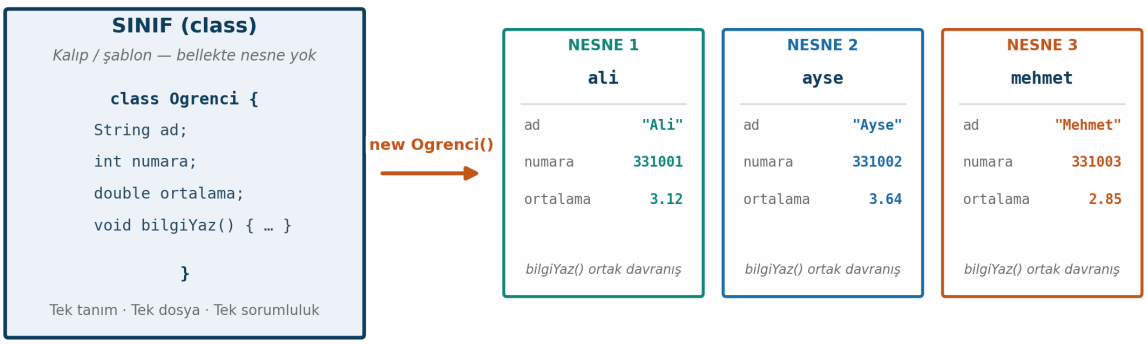
6.1. Sınıf (Class)

Sınıf Belirli bir varlık türünün hangi bilgileri taşıyacağını ve neler yapabileceğini tanımlayan **kalıptır**. Sınıf bir tanımdır; bellekte kendi başına bir varlığı yoktur.

Bir sınıfı, bir evin mimari projesine benzetebilirsiniz. Proje evin kaç odası olacağını, duvarların nerede duracağını belirtir; ama projede yaşayamazsınız. Aynı projeden yüzlerce ev inşa edilebilir ve her evin duvar rengi farklı olabilir.

6.2. Nesne (Object)

Nesne Bir sınıftan üretilmiş, bellekte gerçekten yer kaplayan **somut örnektir**. İngilizcede *instance* (örnek) terimi de kullanılır. Nesne üretme işlemine **örnekleme (instantiation)** denir.



Sınıf birdir; nesne sayısı sınırsızdır. Her nesnenin kendi alan değerleri vardır, davranış tanımı ortaktır.

Şekil 1.7 — Sınıf bir kalıptır; nesneler o kalıptan üretilen somut örneklerdir

6.3. Özellik (Attribute / Field)

Özellik Nesnenin **durumunu** (state) tutan değişkenlerdir. Türkçede *alan* ya da *üye değişken*, İngilizcede *field*, *attribute* ya da *member variable* terimleri kullanılır. Her nesnenin kendi özellik değerleri vardır.

6.4. Metot (Method)

Metot Nesnenin **davranışını** (behaviour) tanımlayan işlevlerdir. Metotlar nesnenin durumunu okuyabilir, değiştirebilir ve dış dünyaya sonuç döndürebilir. Metot tanımı tüm nesneler için ortaktır; ancak metot çalıştığında hangi nesne üzerinde çağrıldıysa onun verileriyle çalışır.

BİLGİ

> Kısa bir bellek yardımı: **Özellik = nesnenin bildikleri. Metot = nesnenin yapabildikleri.** Bir öğrenci nesnesi adını *bilir*; ortalamasını *hesaplayabilir*.

6.5. İlk Sınıfımız: `Ogrenci`

src/Ogrenci.java

```java
public class Ogrenci {

    // --- Ozellikler (attributes / fields) ---
    public String ad;
    public String soyad;
    public int numara;
    public double vizeNotu;
    public double finalNotu;

    // --- Davranislar (behaviours / methods) ---

    /** Vize %40, final %60 agirlikli ders notunu hesaplar. */
    public double ortalamaHesapla() {
        return vizeNotu * 0.4 + finalNotu * 0.6;
    }

    /** Ortalama 50 ve uzerindeyse ogrenci gecmis sayilir. */
    public boolean gectiMi() {
        return ortalamaHesapla() >= 50.0;
    }

    /** Ogrencinin bilgilerini duzenli bicimde ekrana yazar. */
    public void bilgiYaz() {
        System.out.printf("%-8d %-10s %-10s  vize:%5.1f  final:%5.1f  ort:%6.2f  %s%n",
                numara, ad, soyad, vizeNotu, finalNotu,
                ortalamaHesapla(), gectiMi() ? "GECTI" : "KALDI");
    }
}
```

Kod 1.14 — Özellikler ve metotlardan oluşan bir sınıf

Dikkat edilecek noktalar:

- Sınıfın içinde `main` metodu **yoktur**. Bu sınıf çalıştırılmak için değil, başka bir sınıf tarafından *kullanılmak* için yazılmıştır.
- `ortalamaHesapla` metodu, nesnenin `vizeNotu` ve `finalNotu` alanlarına doğrudan (nokta işareti olmadan) erişir. Çünkü metot zaten o nesnenin içindedir.
- `gectiMi` metodu başka bir metodu (`ortalamaHesapla`) çağırır. Kural tek bir yerde tanımlıdır; ağırlıklar değişirse yalnızca tek bir satır güncellenir.
- Bu aşamada alanlar `public` bırakılmıştır. Hafta 4'te bu tasarımın neden sakıncalı olduğunu ve `private` + *getter/setter* ile nasıl düzeltileceğini göreceğiz.

6.6. Nesne Üretme ve Kullanma

src/OgrenciDemo.java

```java
public class OgrenciDemo {

 public static void main(String[] args){

  // 1) Nesne olusturma: "new" ile obekte yer ayrilir, referans degiskene atanir.
  Ogrenci o1 = new Ogrenci();
       o1.ad = "Ali";
       o1.soyad = "Yilmaz";
       o1.numara = 331001;
       o1.vizeNotu = 65;
       o1.finalNotu = 78;

  Ogrenci o2 = new Ogrenci();
       o2.ad = "Ayse";
       o2.soyad = "Demir";
       o2.numara = 331002;
       o2.vizeNotu = 40;
       o2.finalNotu = 52;

  // 2) Her nesnenin kendi alan degerleri vardir; davranis tanimi ortaktir.
  System.out.println("--- Ogrenci Listesi ---");
       o1.bilgiYaz();
       o2.bilgiYaz();

  // 3) Nesneler bir dizide toplanabilir.
  Ogrenci[] sinif = { o1, o2 };
  double toplam = 0;
  for(Ogrenci o : sinif){
           toplam += o.ortalamaHesapla();
  }
  System.out.printf("Sinif ortalamasi: %.2f%n", toplam / sinif.length);
```

Kod 1.15 — Nesne üretimi ve kullanımı

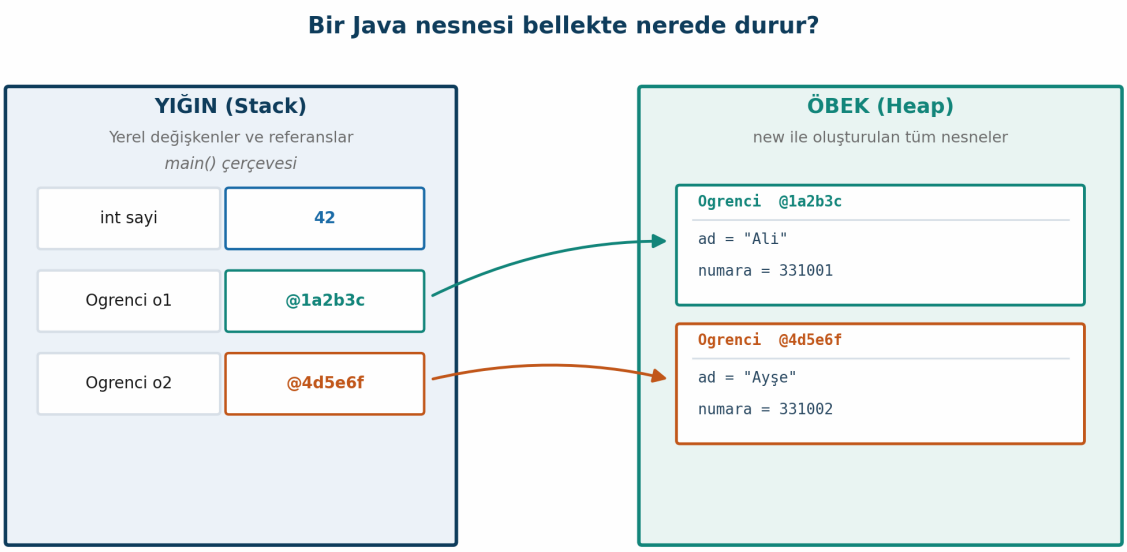
```
▸ TERMINAL — java -cp bin OgrenciDemo
--- Ogrenci Listesi ---
331001   Ali        Yilmaz      vize: 65.0  final: 78.0  ort: 72.80  GECTI
331002   Ayse       Demir       vize: 40.0  final: 52.0  ort: 47.20  KALDI
Sinif ortalamasi: 60.00
```

6.6.1. `new` Anahtar Sözcüğü ve Bellek

`Ogrenci o1 = new Ogrenci();` satırı aslında üç iş yapar:

1. `new Ogrenci()` ifadesi **öbek (heap)** bölgesinde yeni bir `Ogrenci` nesnesi için yer ayırır ve tüm alanlarını varsayılan değerlerle doldurur (sayısal alanlar `0`, `boolean` alanlar `false`, referanslar `null`).
2. Nesnenin bellekteki adresi (referansı) üretilir.

3. Bu referans, **yığın (stack)** bölgesinde tutulan `o1` değişkenine atanır.



Şekil 1.8 — Referans yığında, nesne öbekte tutulur

> **C DİLİNDEN JAVA'YA KÖPRÜ**
>
> C'de `Ogrenci *o1 = malloc(sizeof(Ogrenci));` yazardınız ve işiniz bitince `free(o1);` çağırırdınız. Java'da `free` yoktur: bir nesneye artık hiçbir referans kalmadığında **çöp toplayıcı** o belleği otomatik olarak geri kazanır. Ayrıca `o1->ad` yerine `o1.ad` yazılır; Java'da `->` işleci yoktur.

6.6.2. Referans Semantiği

Java'da nesne değişkenleri nesnenin kendisini değil, **adresini** tutar. Bu, iki değişkenin aynı nesneyi gösterebileceği anlamına gelir:

```
●src/OgrenciDemo.java
1   // 4) Referans semantigi: iki degisken ayni nesneyi gosterebilir.
2   Ogrenci takma = o1;
3        takma.vizeNotu = 90;
4   System.out.println("--- takma.vizeNotu = 90 atamasindan sonra ---");
5        o1.bilgiYaz();    // o1 de degisti, cunku ayni nesnedir.
6   System.out.println("o1 == takma  -> " + (o1 == takma));
```

Kod 1.16 — İki referans, tek nesne

```
▸ TERMINAL — Referans semantiği
--- takma.vizeNotu = 90 atamasindan sonra ---
331001   Ali        Yilmaz      vize: 90.0  final: 78.0  ort: 82.80  GECTI
o1 == takma  -> true
```

> **DİKKAT**

`Ogrenci takma = o1;` satırı **yeni bir nesne oluşturmaz**; yalnızca aynı adresi ikinci bir değişkene kopyalar. Gerçekten bağımsız bir kopya istiyorsanız `new` ile yeni bir nesne üretip alanları tek tek kopyalamanız gerekir. Bu konuya Hafta 5'te yapıcı metotlarla birlikte döneceğiz.

6.6.3. `null` Değeri

Bir referans değişkeni henüz hiçbir nesneyi göstermiyorsa değeri `null`'dır. `null` bir referans üzerinden metot çağırmaya çalışmak, Java'da en sık karşılaşılan çalışma zamanı hatası olan `NullPointerException`'a yol açar.

NullOrnegi.java

```java
Ogrenci o3 = null;
System.out.println(o3 == null); // true
// o3.bilgiYaz();                // NullPointerException!

// Guvenli kullanim:
if(o3 != null){
    o3.bilgiYaz();
}
```

Kod 1.17 — `null` denetimi

6.7. Nesne Dizileri

Nesneler de tıpkı ilkel türler gibi dizilerde saklanabilir. Ancak önemli bir ayrım vardır: `new Ogrenci[3]` ifadesi **üç nesne değil, üç referans için yer** ayırır. Referansların tamamı başlangıçta `null`'dır; her birini ayrıca oluşturmanız gerekir.

NesneDizisi.java

```java
Ogrenci[] sinif = new Ogrenci[3]; // 3 adet null referans olusur
// sinif[0].bilgiYaz();           // NullPointerException!

sinif[0] = new Ogrenci();    // simdi gercek nesne var
sinif[0].ad = "Ali";

// Hazir nesnelerden dizi olusturmak:
Ogrenci[] kucukSinif = { o1, o2 };

double toplam = 0;
for(Ogrenci o : kucukSinif){
    toplam += o.ortalamaHesapla();
}
System.out.printf("Sinif ortalamasi: %.2f%n", toplam / kucukSinif.length);
```

Kod 1.18 — Nesne dizileri

6.8. İkinci Örnek: `Dikdortgen`

Özellik–metot ayrımını pekiştirmek için ikinci bir sınıfa bakalım. Bu örnekte iki tür metot bir arada bulunuyor: **değer döndüren** metotlar (`alan`, `cevre`, `karedirMi`) nesnenin durumunu okur ama değiştirmez; **durum değiştiren** metot (`olcekle`) ise nesnenin alanlarını günceller ve geriye bir şey döndürmez.

src/Dikdortgen.java

```java
public class Dikdortgen {

 public double genislik;   // ozellik
 public double yukseklik;  // ozellik

 public double alan(){      // davranis
  return genislik * yukseklik;
 }

 public double cevre(){      // davranis
  return 2 * (genislik + yukseklik);
 }

 public boolean karedirMi(){    // davranis
  return genislik == yukseklik;
 }

 public void olcekle(double katsayi){ // durumu degistiren davranis
        genislik *= katsayi;
        yukseklik *= katsayi;
 }
}
```

Kod 1.19 — Değer döndüren ve durum değiştiren metotlar

```
▸ TERMINAL — java -cp bin DikdortgenDemo
d1 -> 4.0 x 3.0 | alan=12.00 | cevre=14.00 | kare mi? hayir
d2 -> 5.0 x 5.0 | alan=25.00 | cevre=20.00 | kare mi? evet
d1 iki katina olceklendi:
d1 -> 8.0 x 6.0 | alan=48.00 | cevre=28.00 | kare mi? hayir
```

6.9. Kütüphane Sınıfları da Nesnedir: `Scanner`

Klavyeden veri okumak için kullanılan `Scanner` de bir sınıftır. C'deki `scanf` bir fonksiyondu; Java'da önce bir `Scanner` **nesnesi** oluşturur, sonra bu nesnenin metotlarını çağırırsınız. Bu, standart kütüphanenin de nesne yönelimli tasarlandığını gösteren ilk örnektir.

src/GirdiOrnegi.java

```java
import java.util.Scanner;

public class GirdiOrnegi {

 public static void main(String[] args){
```

```
6    Scanner tarayici = new Scanner(System.in);
7
8    System.out.print("Adiniz: ");
9    String ad = tarayici.nextLine();
10
11   System.out.print("Dogum yiliniz: ");
12   int yil = tarayici.nextInt();
13
14   int yas = 2026 - yil;
15   System.out.printf("Merhaba %s, bu yil %d yasindasiniz.%n", ad, yas);
16
17         tarayici.close(); // Acilan kaynak kapatilir.
18   }
19 }
```

Kod 1.20 — Klavyeden veri okuma

> **DİKKAT**
>
> `nextDouble()` metodu ondalık ayracını işletim sisteminin bölge ayarından alır. Türkçe Windows'ta **virgül** (`3,14`), İngilizce ayarda **nokta** (`3.14`) beklenir. Bu belirsizliği ortadan kaldırmak için `tarayici.useLocale(Locale.US);` yazarak ayracı noktaya sabitleyebilirsiniz.

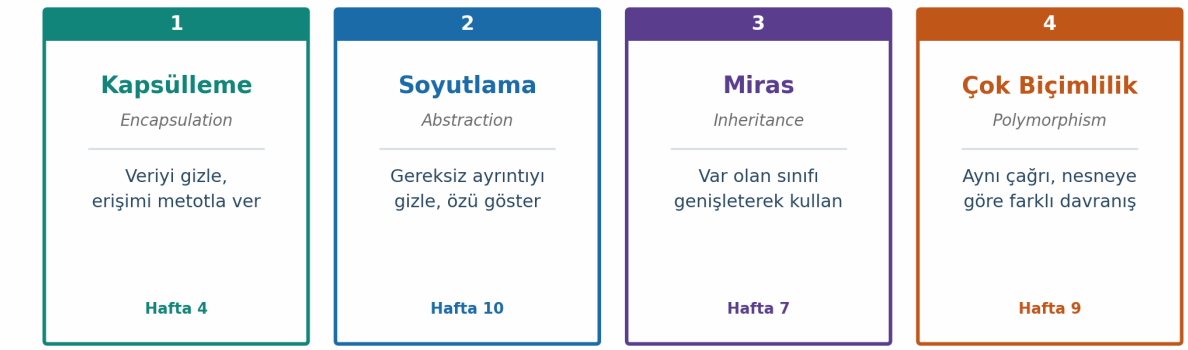
6.10. Kavram Sözlüğü

Türkçe	İngilizce	Kısa tanım
Sınıf	class	Nesnelerin üretildiği kalıp / tanım
Nesne, örnek	object, instance	Sınıftan üretilmiş somut varlık
Örnekleme	instantiation	`new` ile nesne üretme işlemi
Özellik, alan	field, attribute	Nesnenin durumunu tutan değişken
Metot	method	Nesnenin davranışını tanımlayan işlev
Referans	reference	Nesnenin bellekteki adresini tutan değer
Yığın	stack	Yerel değişkenlerin ve referansların tutulduğu bölge
Öbek	heap	`new` ile üretilen nesnelerin tutulduğu bölge
Çöp toplayıcı	garbage collector	Erişilemez nesnelerin belleğini geri kazanan bileşen
Bytecode	bytecode	`javac`'ın ürettiği, JVM'in çalıştırdığı ara kod
Giriş noktası	entry point	Programın başladığı `main` metodu

Tablo 1.9 — Bu haftanın terimleri

7. Dört Temel İlkeye Genel Bakış

Nesne yönelimli programlama dört temel ilke üzerine kuruludur. Bu hafta yalnızca tanışıyoruz; her biri ilerleyen haftalarda ayrıntılı olarak işlenecektir.



Şekil 1.9 — Nesne yönelimli programlamanın dört temel ilkesi ve işlenecekleri haftalar

Kapsülleme (Encapsulation) Verinin dış dünyadan gizlenmesi ve ona yalnızca denetimli metotlarla erişilmesidir. Bölüm 2.2.2'deki `private double bakiye;` satırı bir kapsülleme örneğidir. *(Hafta 4)*

Soyutlama (Abstraction) Bir varlığın kullanıcıyı ilgilendirmeyen ayrıntılarının gizlenip yalnızca gerekli arayüzün açılmasıdır. `paraCek` metodunu çağırırken bakiyenin nasıl güncellendiğini bilmeniz gerekmez. *(Hafta 10)*

Miras (Inheritance) Var olan bir sınıfın özelliklerini ve davranışlarını devralarak yeni bir sınıf türetmektir. `VadeliHesap`, `Hesap` sınıfından türetilebilir. *(Hafta 7)*

Çok Biçimlilik (Polymorphism) Aynı metot çağrısının, çağrıldığı nesnenin gerçek türüne göre farklı davranmasıdır. `hesap.faizIsle()` çağrısı vadeli ve vadesiz hesaplarda farklı çalışabilir. *(Hafta 9)*

8. Hata Türleri ve Hata Ayıklama

Java'da karşılaşacağınız hatalar iki büyük gruba ayrılır. Bir hatayı çözmenin ilk adımı, hangi gruba ait olduğunu anlamaktır.



Şekil 1.10 — Derleme zamanı ve çalışma zamanı hatalarının ayrımı

8.1. Derleme Zamanı Hataları

`javac` kodu bytecode'a çeviremediğinde oluşur. Program **hiç çalışmaz**; `.class` dosyası üretilmez. Hata iletisinde dosya adı, satır numarası ve sorunun türü belirtilir. İyi haber: bu hatalar en ucuz hatalardır, çünkü kullanıcıya hiç ulaşmazlar.

Hata iletisi	Anlamı ve çözümü
`';' expected`	Deyimin sonunda noktalı virgül eksik
`cannot find symbol`	Tanımlanmamış değişken/metot kullanılmış; yazım hatası ya da eksik `import`
`incompatible types: String cannot be converted to int`	Tür uyuşmazlığı; dönüşüm ya da doğru tür kullanımı gerekir
`class X is public, should be declared in a file named X.java`	Dosya adı ile `public` sınıf adı farklı
`variable x might not have been initialized`	Yerel değişkene değer atanmadan kullanılmış
`missing return statement`	Değer döndüren metodun bazı yollarında `return` yok

Tablo 1.10 — Sık karşılaşılan derleme hataları

8.2. Çalışma Zamanı Hataları

Program derlenmiş ve başlamıştır; ancak çalışırken geçersiz bir işlem yapılmıştır. Java bu durumda bir **istisna (exception)** fırlatır ve program durur. Hata iletisinin altındaki **yığın izi (stack trace)** hatanın hangi satırda oluştuğunu gösterir.

İstisna	Nedeni
`NullPointerException`	`null` bir referans üzerinden alan/metot erişimi
`ArrayIndexOutOfBoundsException`	Dizi sınırları dışında indis kullanımı
`ArithmeticException: / by zero`	Tam sayı bölmesinde sıfıra bölme
`InputMismatchException`	`Scanner` beklenen türde veri okuyamadı
`NumberFormatException`	Sayıya çevrilemeyen metin dönüştürülmeye çalışıldı

Tablo 1.11 — Sık karşılaşılan çalışma zamanı istisnaları

> **BİLGİ**
>
> İstisnaların yakalanması ve yönetilmesi Hafta 11'in konusudur. Şimdilik hata iletisini okumayı ve yığın izindeki **kendi dosyanıza ait** satırı bulmayı öğrenmeniz yeterlidir.

8.3. Mantık Hataları

Program derlenir, çalışır ve çökmez; ancak **yanlış sonuç üretir**. Bunlar en sinsi hatalardır, çünkü derleyici ve JVM sizi uyarmaz. Örneğin ortalama hesabında tam sayı bölmesi kullanmak böyle bir hatadır. Bu hataları yakalamanın tek yolu **test etmektir**: normal durumların yanı sıra sınır durumlarını (boş dizi, sıfır, negatif değer, çok büyük değer) da denemelisiniz.

9. Kod Yazım Standartları

Java topluluğunun benimsediği adlandırma kuralları vardır. Bu kurallar derleyici tarafından zorunlu tutulmaz; ancak **dönem boyunca değerlendirmeye dâhildir** ve profesyonel yazılım geliştirmede uyulması beklenir.

Öğe	Kural	Doğru örnek	Yanlış örnek
Sınıf	PascalCase (her sözcük büyük harfle)	`OgrenciKayit`	`ogrenci_kayit`
Metot	camelCase, fiil ile başlar	`ortalamaHesapla()`	`OrtalamaHesapla()`
Değişken / alan	camelCase, ad ile başlar	`vizeNotu`	`Vize_Notu`
Sabit	TÜMÜ_BÜYÜK, alt çizgi ile	`MAKS_OGRENCI`	`maksOgrenci`
Paket	tümü küçük harf	`ktu.oop.hafta01`	`KTU.OOP`
Boolean metot	`is`/`has` ya da `-mi` eki	`gectiMi()`	`gecti()`

Tablo 1.12 — Java adlandırma kuralları

- Girinti için **4 boşluk** kullanın; sekme karakteri yerine boşluk tercih edin (VS Code varsayılan olarak bunu yapar).
- Açılış süslü parantezini satırın sonuna, kapanışı ayrı satıra yazın.
- Değişken adlarında Türkçe karakter (ç, ğ, ı, ö, ş, ü) kullanmayın. Java buna izin verir, ancak taşınabilirlik sorunlarına yol açar. `ogrenciSayisi` yazın, `öğrenciSayısı` yazmayın.
- Bir metot tek bir işi yapsın; adı ne yaptığını açıkça anlatsın.
- Yorumlar *ne yaptığını* değil, **neden yaptığını** açıklasın. `i++ // i'yi artir` yorumu değersizdir.

> **İPUCU**
>
> VS Code'da `Shift+Alt+F` (macOS: `Shift+Option+F`) kısayolu kodu otomatik biçimlendirir. Teslim etmeden önce bu kısayolu kullanmayı alışkanlık hâline getirin.

10. Haftanın Özeti

- Yapısal programlamada **veri ile işlem ayrıdır**; nesne yönelimli programlamada **birlikte** tutulur. Bu, verinin korunmasını ve kuralların tek yerde toplanmasını sağlar.
- Java kaynak kodu `javac` ile **bytecode**'a derlenir; bytecode **JVM** üzerinde çalışır. Bu sayede aynı derlenmiş program her platformda çalışabilir.
- **JDK** geliştirme araçlarını, **JRE** çalıştırma ortamını, **JVM** ise sanal makineyi ifade eder. Bu derste JDK gereklidir.
- Java'da her kod bir **sınıfın** içindedir. Program `public static void main(String[] args)` metodundan başlar.
- `public` bir sınıfın adı, dosya adıyla **birebir aynı** olmalıdır.
- **Sınıf** bir kalıptır; **nesne** o kalıptan üretilen somut örnektir. **Özellik** nesnenin bildiklerini, **metot** yapabildiklerini tanımlar.
- `new` ile nesne **öbekte** oluşturulur; değişken ise **yığında** duran bir **referans** tutar. İki değişken aynı nesneyi gösterebilir.
- **Derleme zamanı** hataları programın çalışmasını engeller; **çalışma zamanı** hataları program çalışırken ortaya çıkar; **mantık hataları** ise yalnızca test ile yakalanır.

→ BİR SONRAKİ HAFTA

Sınıflar ve Nesneler. Yapıcı (constructor) metotlar, `this` anahtar sözcüğü, nesne yaşam döngüsü ve kapsülleme kavramına giriş. Bu hafta `public` bıraktığımız alanları `private` yapmaya başlayacağız.

11. Alıştırmalar

Zorluk göstergesi: temel · orta · ileri

11.1. Kavram Soruları

Alıştırma 1

Bir arkadaşınıza sınıf ile nesne arasındaki farkı, programlama terimi kullanmadan, günlük hayattan bir benzetmeyle anlatın. Benzetmenizi üç cümleyle yazın.

Alıştırma 2

Aşağıdaki ifadelerin hangileri **özellik**, hangileri **metot** olarak modellenmelidir? Gerekçesini yazın: *(a)* bir otomobilin rengi, *(b)* bir otomobilin hızlanması, *(c)* bir kitabın sayfa sayısı, *(d)* bir kitabın ödünç verilmesi, *(e)* bir öğrencinin not ortalaması.

İpucu: Ortalama örneğinde iki farklı tasarım mümkündür; her ikisinin de artı ve eksilerini tartışın.

Alıştırma 3

Bölüm 2.2.1'deki yapısal çözümde `sahipler` ve `bakiyeler` dizileri kullanılmıştı. Bu iki diziden yalnızca birinin sıralanması hâlinde ne olur? Nesne yönelimli çözümde aynı sorun neden oluşmaz?

Alıştırma 4

JDK, JRE ve JVM kavramlarını birer cümleyle tanımlayın. Bir kullanıcının bilgisayarında yalnızca JRE varsa, sizin yazdığınız programı çalıştırabilir mi? Peki derleyebilir mi?

11.2. Kod Okuma ve Hata Bulma

Alıştırma 5

Aşağıdaki kodda **dört** ayrı hata vardır. Hepsini bulun, her birinin derleme zamanı hatası mı yoksa çalışma zamanı hatası mı olduğunu belirtin ve düzeltilmiş kodu yazın.

Merhaba.java

```
1 public class merhaba {
2   public void main(String[] args){
3     int[] notlar = {70, 85, 90}
4     System.out.println("Ortalama: " + notlar[3]);
```

```
    }
}
```

Alıştırma 5 için kod

Alıştırma 6

Aşağıdaki programın ekrana ne yazdıracağını **çalıştırmadan** tahmin edin ve gerekçesini yazın. Sonra çalıştırıp tahmininizi doğrulayın.

Alistirma6.java

```
Dikdortgen a = new Dikdortgen();
a.genislik = 3; a.yukseklik = 4;

Dikdortgen b = a;
b.genislik = 10;

Dikdortgen c = new Dikdortgen();
c.genislik = a.genislik; c.yukseklik = a.yukseklik;
c.olcekle(2);

System.out.println(a.alan());
System.out.println(b.alan());
System.out.println(c.alan());
System.out.println(a == b);
System.out.println(a == c);
```

Alıştırma 6 için kod

11.3. Programlama Alıştırmaları

Alıştırma 7

`Daire` adında bir sınıf yazın. Özellik: `yaricap`. Metotlar: `alan()`, `cevre()`, `capiHesapla()` ve verilen katsayıyla yarıçapı büyüten `buyut(double katsayi)`. Ayrıca `DaireDemo` sınıfında üç farklı daire oluşturup bilgilerini biçimli olarak yazdırın.

İpucu: π sabiti için `Math.PI` kullanın.

Alıştırma 8

`Calisan` adında bir sınıf yazın. Özellikler: `ad`, `unvan`, `brutMaas`, `calismaYili`. Metotlar: `netMaas()` (brütten %15 kesinti), `kidemliMi()` (10 yıldan fazla çalışmışsa `true`), `zamYap(double yuzde)`. Beş çalışandan oluşan bir dizi oluşturup toplam net maaş giderini ve en kıdemli çalışanı bulun.

Alıştırma 9

Bölüm 2.2.1'deki `YapisalYaklasim` programını inceleyin. Aynı işlevselliği bir `Hesap` sınıfı kullanarak yeniden

yazın; ancak **bu kez `paraCek` metodunda 5 TL işlem ücreti** kesilsin. Bu değişikliği yapmak iki tasarımda kaç dosyaya dokunmanızı gerektiriyor? Karşılaştırın.

İpucu: Bu alıştırmanın asıl amacı kod yazmak değil, **değişikliğin maliyetini** gözlemlemektir.

Alıştırma 10

`Sinav` adında bir sınıf tasarlayın: bir sınavın dersi, tarihi, süresi (dakika) ve soru sayısı olsun. `soruBasinaSure()` metodu ortalama süreyi dakika cinsinden döndürsün. `Sinav` nesnelerinden oluşan bir dizi üzerinde en uzun sınavı bulan bir metot yazın. Sınıfınızı tasarlarken hangi bilgileri özellik, hangilerini metot yaptığınızı **bir paragrafla gerekçelendirin**.

11.4. Öz Değerlendirme

Öz Değerlendirme Kontrol Listesi

JDK'yı kurdum; `java -version` ve `javac -version` komutları çalışıyor.

VS Code'a Extension Pack for Java uzantısını kurdum.

Bir Java programını terminalden `javac` ve `java` komutlarıyla derleyip çalıştırabiliyorum.

`main` metodunun imzasını ezberden ve hatasız yazabiliyorum.

Sınıf ile nesne arasındaki farkı kendi cümlelerimle açıklayabiliyorum.

Özellik ile metot arasındaki farkı örnek vererek anlatabiliyorum.

`new` ile nesne oluşturup alanlarına değer atayabiliyor ve metotlarını çağırabiliyorum.

İki referansın aynı nesneyi göstermesinin sonuçlarını açıklayabiliyorum.

Derleme zamanı hatası ile çalışma zamanı hatasını ayırt edebiliyorum.

Java adlandırma kurallarına uygun kod yazıyorum.

12. Kaynaklar

12.1. Ders Kitapları

- Deitel, P. & Deitel, H. *Java How to Program (Early Objects)*, 11. baskı, Pearson. — Bölüm 1–3.
- Horstmann, C. S. *Core Java, Volume I: Fundamentals*, 12. baskı, Oracle Press. — Bölüm 1–3.
- Schildt, H. *Java: The Complete Reference*, 12. baskı, McGraw-Hill. — Bölüm 1–2.
- Bloch, J. *Effective Java*, 3. baskı, Addison-Wesley. — İleri okuma; dönem sonuna doğru başvurulması önerilir.

12.2. Çevrimiçi Kaynaklar

- Oracle Java Tutorials — *docs.oracle.com/javase/tutorial*
- Java SE 21 API Belgeleri — *docs.oracle.com/en/java/javase/21/docs/api*
- Eclipse Temurin JDK indirme — *adoptium.net*
- VS Code Java belgeleri — *code.visualstudio.com/docs/languages/java*

12.3. İletişim

	Öğretim Elemanı	Laboratuvar Sorumlusu
Ad	Dr. Öğr. Üyesi Rıfat BENVENİSTE	Arş. Gör. Betül MUMCU
Sorumluluk	Teorik ders, sınavlar, proje danışmanlığı	Laboratuvar uygulamaları, ödev teslimi, föy soruları
Görüşme	Ofis saatlerinde	Laboratuvar saatlerinde